

SECTION 6: FUNCTIONS, SUBROUTINES, MODULES

Break large codes down into procedures such as subroutines and functions.

Lecture 21: Functions

(or function subprograms)

General Form:

```
FUNCTION function_name (argument-list)
```

```
!Declaration Section (must declare type of function-name)
```

```
REAL :: function_name, ...
```

```
!Execution Section
```

```
    statements...
```

```
function_name = a result
```

```
!Termination Section
```

```
RETURN
```

```
END FUNCTION function_name
```

Example:

A user defined external function to convert degrees F to degrees C

```

PROGRAM TEST_FUNCTION

  IMPLICIT NONE

  REAL :: CONVERT_C_TO_F, degrees_C, degrees_F
  INTEGER :: i, year=1900

  PRINT*

  DO i=0,100,10
    degrees_C=REAL(i)
    degrees_F=CONVERT_C_TO_F(degrees_C)
    WRITE(*,'(F6.0,3X,F6.0)')degrees_C,degrees_F
    PRINT*
  END DO

  END PROGRAM TEST_FUNCTION

!*****

!   EXTERNAL FUNCTION

  FUNCTION CONVERT_C_TO_F(degrees_C)
    REAL :: CONVERT_C_TO_F
    REAL, INTENT(IN) :: degrees_C

!    $T(^{\circ}\text{F}) = T(^{\circ}\text{C}) \times 1.8 + 32$ 

    CONVERT_C_TO_F=degrees_C*1.8 + 32.0

    RETURN

  END FUNCTION CONVERT_C_TO_F

```

```
cmd.exe - Shortcut
G:\FORTRAN_COURSE\SECTION6-FUNCTIONS_SUBS_MODS\CODES>a.exe
    0.      32.
   10.     50.
   20.     68.
   30.     86.
   40.    104.
   50.    122.
   60.    140.
   70.    158.
   80.    176.
   90.    194.
  100.    212.

G:\FORTRAN_COURSE\SECTION6-FUNCTIONS_SUBS_MODS\CODES>
```

Example as an Internal Function

```
PROGRAM TEST_FUNCTION

  IMPLICIT NONE

  REAL :: degrees_C, degrees_F
  INTEGER :: i, year=1900

  PRINT*

  DO i=0,100,10

    degrees_C=REAL(i)
    degrees_F=CONVERT_C_TO_F(degrees_C)
    WRITE(*,'(F6.0,3X,F6.0)')degrees_C,degrees_F
    PRINT*

  END DO

! *****

CONTAINS

!   INTERNAL FUNCTION
  FUNCTION CONVERT_C_TO_F(degrees_C)
    REAL :: CONVERT_C_TO_F
    REAL, INTENT(IN) :: degrees_C
!    $T(^{\circ}\text{F}) = T(^{\circ}\text{C}) \times 1.8 + 32$ 
    CONVERT_C_TO_F=degrees_C*1.8 + 32.0
    RETURN
  END FUNCTION CONVERT_C_TO_F

! *****

END PROGRAM TEST_FUNCTION
```

Lecture 22: Subroutines

Often designed to return more than 1 value

Values are returned through arguments rather than function names

Subroutine referenced through a CALL statement

General Format:

Subroutine *subroutine_name* (*argument list*)

! Declaration Section

! Execution Section

Statements ...

! Termination Section

RETURN

END SUBROUTINE *subroutine_name*

Example: Use subroutine to compute solution to Laplace equation.

```

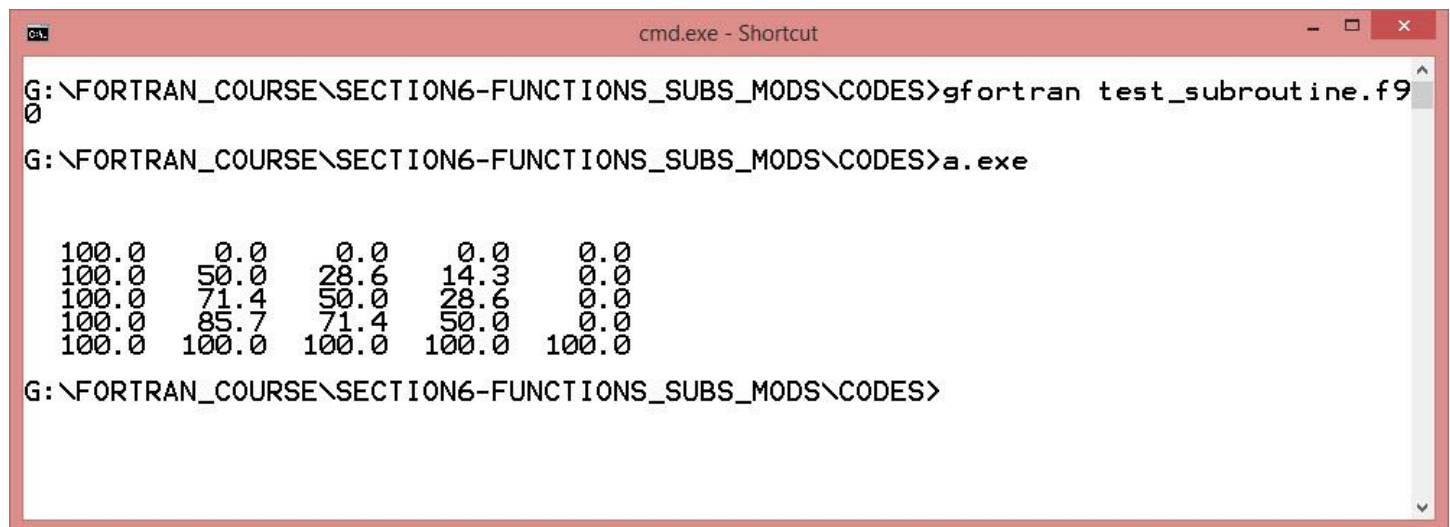
!   EXAMPLE SUBROUTINE CALL
!   SOLVE LAPLACE EQUATION
PROGRAM test_subroutine
IMPLICIT NONE
INTEGER, PARAMETER :: NX=5,NY=5
INTEGER :: i,j,max_iter=100
REAL, DIMENSION(NX,NY) :: phi=0

phi(1,:)=100.
phi(:,1)=100.

CALL LAPLACE(NX,NY,max_iter,phi)

PRINT*
DO j=NY,1,-1
    WRITE(*,50) (PHI(i,j),i=1,NX)
END DO
50  FORMAT(5f7.1)
END PROGRAM test_subroutine

```



```

cmd.exe - Shortcut
G:\FORTRAN_COURSE\SECTION6-FUNCTIONS_SUBS_MODS\CODES>gfortran test_subroutine.f90
G:\FORTRAN_COURSE\SECTION6-FUNCTIONS_SUBS_MODS\CODES>a.exe
100.0    0.0    0.0    0.0    0.0
100.0   50.0   28.6   14.3    0.0
100.0   71.4   50.0   28.6    0.0
100.0   85.7   71.4   50.0    0.0
100.0  100.0  100.0  100.0  100.0
G:\FORTRAN_COURSE\SECTION6-FUNCTIONS_SUBS_MODS\CODES>

```

```

!*****
SUBROUTINE LAPLACE(n1,n2,max_iter,phi)
IMPLICIT NONE

!   NEED TO DECLARE ALL THE SUBROUTINE ARGUMENTS and
!   ANY OTHER VARIABLES LOCAL TO THE SUBROUTINE

!   part of argument list
INTEGER, INTENT(IN) :: max_iter, n1, n2
REAL, DIMENSION(n1,n2), INTENT(INOUT) :: phi

!   local to subroutine
INTEGER :: i, j, iter

DO iter=1,max_iter
  DO i=2,n1-1
    DO j=2,n2-1
      phi(i,j)=0.25*(phi(i+1,j)+phi(i-1,j)+ &
                    phi(i,j+1)+phi(i,j-1))
    END DO
  END DO
END DO

RETURN
END SUBROUTINE LAPLACE
!*****

```

Lecture 23: Modules

Used to group together typically functions, subroutines, and/or type declarations.

For Example:

```
MODULE group_functions

  IMPLICIT NONE

CONTAINS

! *****

  FUNCTION CONVERT_C_TO_F(degrees_C)

    REAL :: CONVERT_C_TO_F

    REAL, INTENT(IN) :: degrees_C

    CONVERT_C_TO_F=degrees_C*1.8 + 32.0

    RETURN

  END FUNCTION CONVERT_C_TO_F

! *****

  FUNCTION FEET_TO_METERS(feet)

    REAL :: FEET_TO_METERS

    REAL, INTENT(IN) :: feet

    FEET_TO_METERS= 0.3048*feet

    RETURN

  END FUNCTION FEET_TO_METERS

! *****

  FUNCTION HYDROSTATIC_PRESSURE(rho,h)

    REAL, PARAMETER :: g=9.91

    REAL :: HYDROSTATIC_PRESSURE

    REAL, INTENT(IN) :: rho, h

    HYDROSTATIC_PRESSURE=rho*g*h
```



```
!*****
```

```
RETURN
```

```
END FUNCTION HYDROSTATIC_PRESSURE
```

```
END MODULE group_functions
```

```
!*****
```

```
PROGRAM test_modules
```

```
USE group_functions
```

```
IMPLICIT NONE
```

```
REAL :: temperature_c, temperature_f, feet, meters, &  
      rho, depth, pressure
```

```
PRINT*
```

```
WRITE(*, '(A)', ADVANCE='NO') 'Input temperature in degrees C: '
```

```
READ(*, *) temperature_c
```

```
temperature_f=CONVERT_C_TO_F(temperature_c)
```

```
WRITE(*, '(A,f6.1)') 'The temperature in degrees  
F:', temperature_f
```

```
PRINT*
```

```
WRITE(*, '(A)', ADVANCE='NO') 'Enter the number of feet: '
```

```
READ(*, *) feet
```

```
meters=FEET_TO_METERS(feet)
```

```
WRITE(*, '(A,f6.3)') 'The equivalent number of meters  
is:', meters
```

```
PRINT*
```

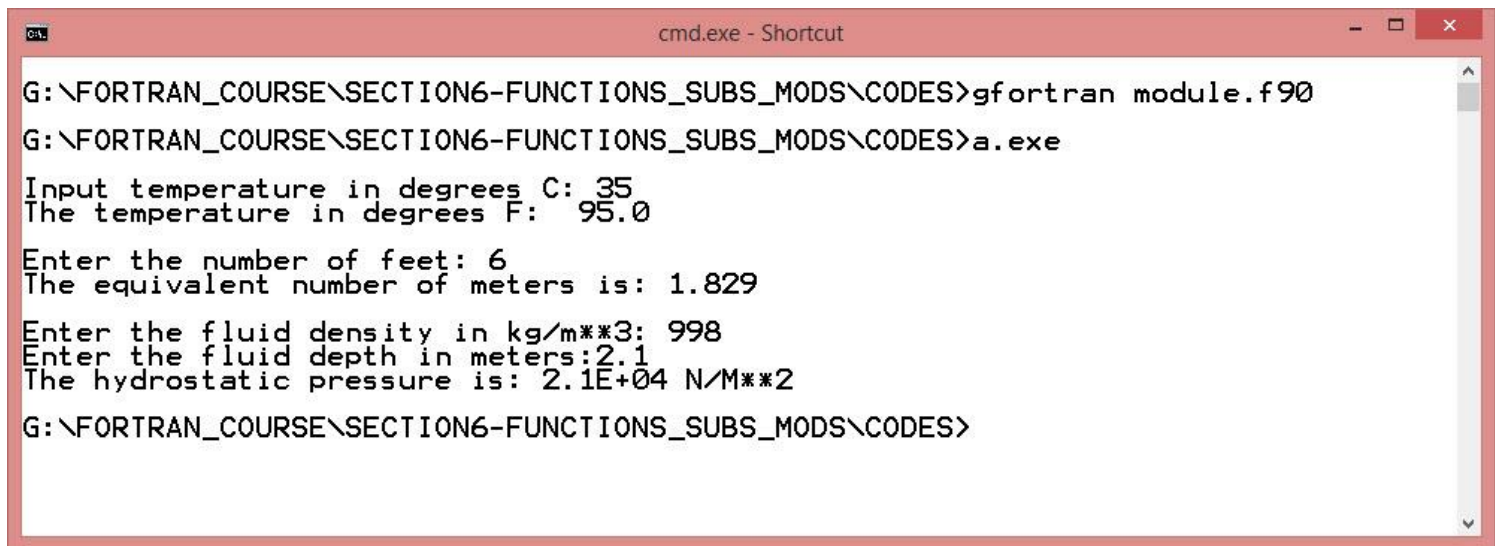
```

WRITE(*, '(A)', ADVANCE='NO') 'Enter the fluid density in
kg/m**3: '
READ(*, *) rho
WRITE(*, '(A)', ADVANCE='NO') 'Enter the fluid depth in meters:'
READ(*, *) depth
pressure=HYDROSTATIC_PRESSURE(rho, depth)
WRITE(*, '(A, ES8.1, A)')    &
    'The hydrostatic pressure is:', pressure, ' N/M**2'

END PROGRAM test_modules

```

OUTPUT:

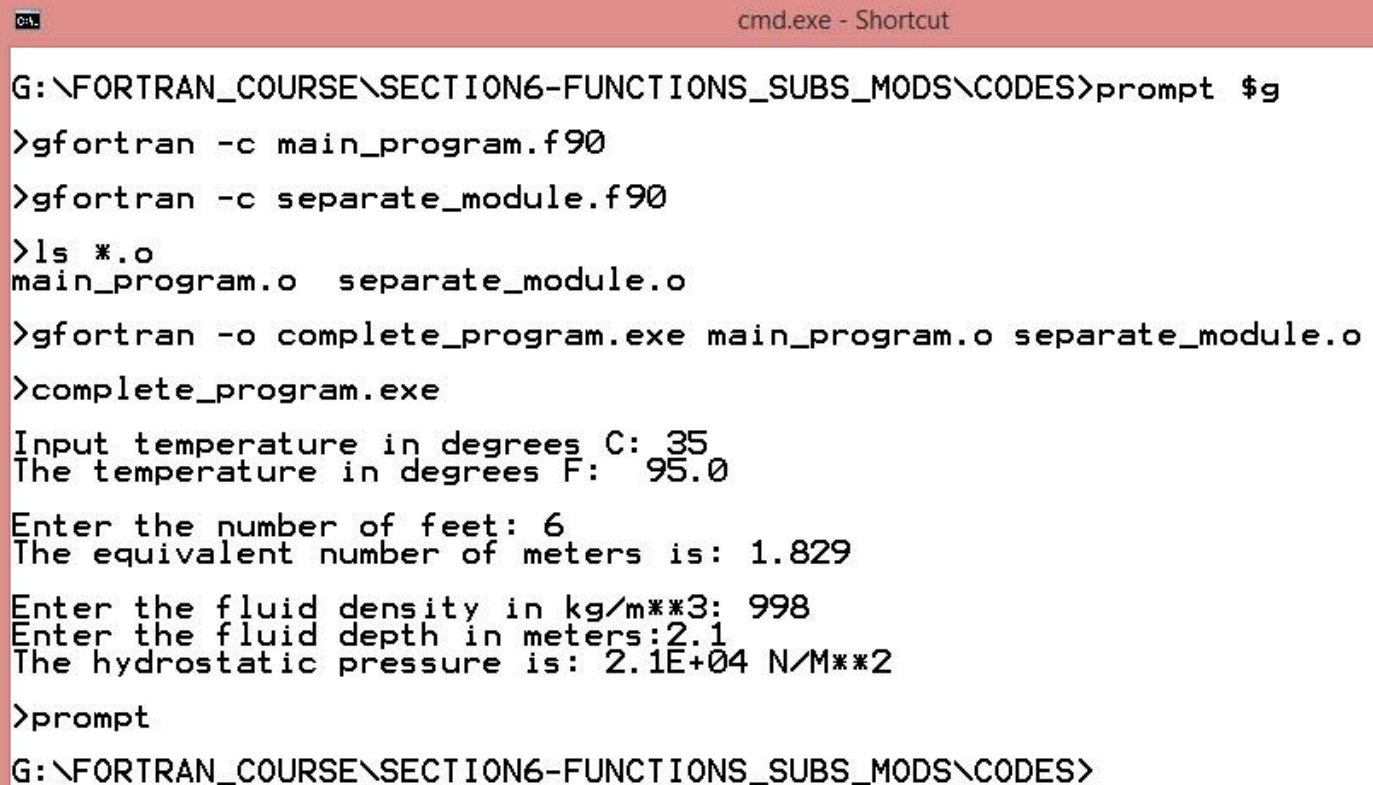


```

cmd.exe - Shortcut
G:\FORTRAN_COURSE\SECTION6-FUNCTIONS_SUBS_MODS\CODES>gfortran module.f90
G:\FORTRAN_COURSE\SECTION6-FUNCTIONS_SUBS_MODS\CODES>a.exe
Input temperature in degrees C: 35
The temperature in degrees F: 95.0
Enter the number of feet: 6
The equivalent number of meters is: 1.829
Enter the fluid density in kg/m**3: 998
Enter the fluid depth in meters: 2.1
The hydrostatic pressure is: 2.1E+04 N/M**2
G:\FORTRAN_COURSE\SECTION6-FUNCTIONS_SUBS_MODS\CODES>

```

An alternative is to compile both the main program and module separately by first creating explicit object files. See the example below.



```
G:\FORTRAN_COURSE\SECTION6-FUNCTIONS_SUBS_MODS\CODES>prompt $g
>gfortran -c main_program.f90
>gfortran -c separate_module.f90
>ls *.o
main_program.o  separate_module.o
>gfortran -o complete_program.exe main_program.o separate_module.o
>complete_program.exe
Input temperature in degrees C: 35
The temperature in degrees F: 95.0
Enter the number of feet: 6
The equivalent number of meters is: 1.829
Enter the fluid density in kg/m**3: 998
Enter the fluid depth in meters: 2.1
The hydrostatic pressure is: 2.1E+04 N/M**2
>prompt
G:\FORTRAN_COURSE\SECTION6-FUNCTIONS_SUBS_MODS\CODES>
```

!ANOTHER EXAMPLE USE FOR MODULES

```
!*****
```

```
MODULE VARIABLES
  IMPLICIT NONE
  SAVE
  INTEGER, PARAMETER :: nx=31, ny=31
  INTEGER :: iteration, number_iterations, iter,
itime, &
          number_time_steps, iprob

  REAL ::
dx, dy, xmax, ymax, rho, mu, omegau, omegav, omegap, error, &
          xx, yy, velmag, min, mout, dt, t=0.0
  REAL, dimension(0:nx+1,0:ny+1) ::
u, v, p, uold, vold, pp, &

uu, vv, pcorner, apu, apv, app, source, ae, aw, an, as, &
          u0, v0
```

```
END MODULE VARIABLE
```

```
!*****
```

```
PROGRAM NAVIERSTOKES
  USE VARIABLES
  IMPLICIT NONE
!  set fluid properties
  mu=.0001333      !kinematic viscosity
  rho=1           !density
  number_iterations=6    !number of outer iterations;
  number_time_steps=2000
  dt=0.1
!  if iprob=1, driven cavity
!  if iprob=2, channel
  iprob=1
```

SUBROUTINES USING "VARIABLES" FOLLOW in ACTUAL CODE